Planejamento de Testes API (Administrador)

1. Apresentação

Este documento apresenta o plano de testes para o módulo Administrador da API Cinema, desenvolvido no Challenge Final do PB AWS & AI for QE da Compass UOL.

O objetivo é assegurar a qualidade e a conformidade da aplicação com os requisitos especificados na User Story US 01 - Gerenciamento de Sistema (Administrador).

Este plano contempla testes manuais e automatizados, empregando Postman para testes exploratórios e Robot Framework para automação.

A gestão dos testes é realizada por meio do QALITY no Jira, com versionamento via Git, utilizando um ambiente local para execução dos testes e da aplicação.

2. Objetivo

O objetivo consiste em validar a qualidade e a conformidade funcional do módulo de administração da API. Tal processo envolve a identificação de defeitos, a mitigação de riscos e a garantia de que a API satisfaz os critérios funcionais de aceitação.

A estratégia adotada combina testes manuais exploratórios com uma suíte de testes automatizados utilizando o Robot Framework. Essa abordagem assegura agilidade operacional e ampla cobertura de regressão, preparando adequadamente para o desafio técnico proposto.

3. Escopo

Não pode faltar:	• Realizar testes completos em todos os endpoints (CRUD) dos módulos de Filmes, Teatros, Sessões e Reservas, direcionados a administradores. • Preparar a massa de dados necessária para os cenários de teste, incluindo a geração dinâmica de dados para automação. • Desenvolver testes automatizados utilizando Robot Framework para cobrir cenários críticos e de regressão. • Gerenciar o ciclo de testes, abrangendo casos de teste, execução, defeitos e questões abertas

	relacionadas a bugs e melhorias, utilizando o QALITY integrado ao Jira. • Concentrar esforços em testes funcionais da API. • Versionar o plano de testes e o código de automação no repositório Git.
É bom ter:	• Validações de responses com JSON Schema para garantir a estrutura dos dados. • Realizar uma rodada de testes manuais exploratórios para validação inicial e descoberta de defeitos.
Fora do escopo:	• Testes de interface do usuário (frontend). • Testes de integração com sistemas externos. • Testes de usabilidade. • Testes de demais módulos da API, excluindo os relacionados à administração.

4. Análise

A estratégia de teste será dividida em fases para garantir uma abordagem estruturada e eficiente:

Fase 1: Testes Manuais e Exploratórios

Ferramenta: Postman.

Objetivo: Realizar a primeira rodada de execução baseada nos casos de teste definidos. O foco é a validação funcional inicial, a descoberta de defeitos óbvios e a compreensão aprofundada do comportamento da API.

Fase 2: Refinamento e Desenvolvimento da Automação

Ferramenta: Robot Framework.

Objetivo: Após a validação manual, os testes candidatos à automação serão desenvolvidos. Esta fase foca em criar uma suite de testes robusta que possa ser executada rapidamente para garantir a regressão.

Fase 3: Execução Automatizada e Relatórios

Ferramenta: Robot Framework, QALity.

Objetivo: Executar a suite de testes automatizados, garantindo que novas alterações não introduzam defeitos. Os resultados serão reportados e gerenciados no QALITY.

5. Técnicas aplicadas

- Testes Baseados em Riscos: Priorização dos testes com base no impacto e probabilidade de falha, focando em funcionalidades críticas como criação de usuário e validações de segurança.
- Testes Exploratórios e Baseados em Sessão: Para complementar a cobertura dos casos de teste e descobrir defeitos não previstos.
- Técnicas de Design de Teste:
  - Particionamento de Equivalência: Para otimizar a seleção de dados de teste (ex: e-mails válidos/inválidos, provedores permitidos/não permitidos).
  - Análise de Valor Limite: Foco nos limites das regras de negócio (ex: senhas com 4, 5, 10 e 11 caracteres).
  - Teste de Tabela de Decisão: Para validar as regras complexas, como o comportamento do PUT para atualizar um usuário.

6. Mapa mental da aplicação

7. Cenários de Testes

- Cenário: a ver... - US01-CE1

ID do Caso de Teste	Descrição	Dados de Entrada	Resultado Esperado	Prioridade
US001-CE1-CT1				

8. Priorização da execução dos cenários de teste

A priorização da execução dos cenários de teste é determinada pela criticidade da funcionalidade, pelo risco associado, pela frequência de uso e pela dependência de outros módulos. A prioridade atribuída a cada caso de teste (Alta, Média, Baixa) encontra-se indicada nas tabelas de cenários.

- Prioridade Alta: Inclui funcionalidades críticas, cenários de alto risco (como questões de segurança e falhas que impedem o uso principal do sistema) e fluxos principais do usuário. Estes devem ser executados prioritariamente e com maior frequência.
- Prioridade Média: Compreende funcionalidades importantes, cenários de risco moderado e fluxos alternativos. A execução deve ocorrer após a conclusão dos testes de alta prioridade.
- Prioridade Baixa: Engloba funcionalidades menos críticas, cenários de baixo risco ou casos de borda com menor probabilidade de ocorrência. Devem ser executados quando houver disponibilidade de tempo e recursos.

9. Matriz de Risco

Risco	Impacto	Probabilidade	Mitigação	Contigência
Falha no cadastro de usuário	Alta	Média	Testes abrangentes de validação de entrada e unicidade de e-mail. Automação	Rollback da versão da API. Correção de emergência.

			de testes de regressão.	
Exposição de dados sensíveis de usuário	Alta	Baixa	Testes de segurança (injeção, autenticação /autorização). Criptografia de dados em trânsito e em repouso.	Notificação de usuários afetados. Análise forense.
Lentidão na listagem de usuários	Média	Média	Testes de desempenho (carga e estresse). Otimização de consultas ao banco de dados.	Otimização de infraestrutura.
Falha na exclusão de usuário com carrinho associado	Média	Baixa	Testes de integridade referencial. Validação de regras de negócio na API.	Correção manual de dados.

10. Cobertura de testes

-

11. Ferramentas, Ambiente e Infraestrutura

- Gerenciamento e Versionamento:
  - Jira com QALITY: Para gestão completa dos casos de teste, planejamento de ciclos e reporte de defeitos.

    - Git: Para controle de versão de toda a documentação (Plano de Testes) e do código-fonte da automação.
- Ferramentas de Teste:
    - Postman: Utilizado para os testes manuais e exploratórios.
    - Robot Framework: Ferramenta principal para o desenvolvimento dos testes automatizados.
- Infraestrutura e Ambiente de Teste:
    - O ambiente será configurado localmente.
    - Configuração: Node.js e NPM para hospedar a API

12. Testes candidatos a automação

Com base na estratégia, os seguintes casos de teste são os principais candidatos para automação com Robot Framework após a validação manual, por representarem o fluxo principal (caminho feliz) e as validações de negócio mais críticas:

- Testes de Caminho Feliz:
    - US01-CE1-CT1: Cadastro de usuário comum.
    - US01-CE2-CT1: Listar usuários.
    - US01-CE2-CT3: Buscar usuário por ID.
    - US01-CE3-CT1: Atualizar usuário existente.
    - US01-CE4-CT1: Excluir usuário.
- Testes de Regras de Negócio Críticas:
    - US01-CE1-CT3: Tentativa de cadastro com e-mail duplicado.
    - US01-CE3-CT3: Criação de usuário via PUT (comportamento específico).
    - US01-CE4-CT3: Tentativa de exclusão de usuário com dependência (carrinho).

13. Cronograma

Atividade	Início	Término
Planejamento de Testes	29 de set. de 2025	30 de set. de 2025
Sessão de Testes	1 de out. de 2025	7 de out. de 2025
Relatório de Testes	8 de out. de 2025	9 de out. de 2025

14. Referências

- Test Coverage Criteria for RESTful Web APIs - Alberto Martin-Lopez, Sergio Segura and Antonio Ruiz-Cortés
- ISTQB Certified Tester Foundation Level (CTFL) 4.0 Syllabus
- ISO/IEC 25010:2011 - System and Software Quality Models
- ISO/IEC/IEEE 29119 - Software Testing